DATABASE DESIGN & IMPLEMENTATION DOCUMENT

NAILVERSE

Not just NAILS.. IT's the WHOLE MOOD

Online Nail Art & Nail Beauty Online Platform

WEEK - 4

4. Appointment and Appointment Details Management System

Scenario

Nailverse needs to maintain information about the appointments booked by customers. A customer can book multiple appointments, and each appointment may include more than one service (for example, a Manicure combined with Nail Art).

To manage this, two new tables are created:

- APPOINTMENT – stores the main information about each booking.
- APPOINTMENT_DETAILS – stores the services included in each appointment.

The APPOINTMENT table is connected with the existing CUSTOMER table (from Week 1). The APPOINTMENT_DETAILS table connects each appointment with the services purchased from the existing SERVICE table (from Week 2).

This structure helps the system maintain customer bookings and generate useful reports about service demand.

Design Appointment and Appointment Details Tables

APPOINTMENT TABLE:

```
SQL> CREATE TABLE Appointment (
  2      Appointment_ID VARCHAR2(7),
  3      Customer_ID VARCHAR2(20),
  4      Appointment_Date DATE,
  5      Total_Amount NUMBER(10,2),
  6      Appointment_Status VARCHAR2(20)
  7  );

Table created.
SQL> desc appointment
 Name                                      Null?    Type
 ----------------------------------------- -------- ------------
---------------
 APPOINTMENT_ID                                     VARCHAR2(7)
 CUSTOMER_ID                                        VARCHAR2(20)
 APPOINTMENT_DATE                                   DATE
 TOTAL_AMOUNT                                       NUMBER(10,2)
 APPOINTMENT_STATUS                                 VARCHAR2(20)
```

APPOINTMENT DETAILS TABLE:

```
SQL> CREATE TABLE Appointment_Details (
  2      Appointment_Detail_ID VARCHAR2(7),
  3      Appointment_ID VARCHAR2(7),
  4      Service_ID VARCHAR2(7),
  5      Quantity NUMBER,
  6      Price NUMBER(10,2)
  7  );

Table created.
SQL> desc Appointment_Details;
 Name                                      Null?    Type
 ----------------------------------------- -------- ------------
----------------
 APPOINTMENT_DETAIL_ID                              VARCHAR2(7)
 APPOINTMENT_ID                                     VARCHAR2(7)
 SERVICE_ID                                         VARCHAR2(7)
 QUANTITY                                           NUMBER
 PRICE                                              NUMBER(10,2)
```

Establish Relationships

The following relationships are established between the tables.

Customer → Appointment

One customer can book many appointments.

Relationship:

CUSTOMER (1) ——————— (M) APPOINTMENT

The CUSTOMER_ID in the APPOINTMENT table refers to the CUSTOMER_ID in the CUSTOMER table.

Appointment → Appointment Details

One appointment can contain multiple appointment details.

APPOINTMENT (1) ——————— (M) APPOINTMENT_DETAILS

Service → Appointment Details

A service can appear in multiple appointment details.

SERVICE (1) ——————— (M) APPOINTMENT_DETAILS

Therefore, the many-to-many relationship between customers' appointments and services is handled using the APPOINTMENT_DETAILS table.

Foreign Key Commands :

```
SQL> ALTER TABLE Appointment ADD CONSTRAINT PK_Appointment PRIMA
RY KEY (Appointment_ID);

Table altered.

SQL> ALTER TABLE Appointment ADD CONSTRAINT FK_Appointment_Custo
mer FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
  2  ;

Table altered.

SQL> ALTER TABLE Appointment_Details ADD CONSTRAINT PK_Appointme
nt_Details PRIMARY KEY (Appointment_Detail_ID);

Table altered.
```

```
SQL> ALTER TABLE Appointment_Details
  2  ADD CONSTRAINT FK_ApptDetails_Appointment
  3  FOREIGN KEY (Appointment_ID)
  4  REFERENCES Appointment(Appointment_ID);

Table altered.

SQL> ALTER TABLE Appointment_Details
  2  ADD CONSTRAINT FK_ApptDetails_Service
  3  FOREIGN KEY (Service_ID)
  4  REFERENCES Service(Service_ID);

Table altered.
```

Apply Constraints:

```
SQL> ALTER TABLE Appointment MODIFY Customer_ID CONSTRAINT NN_Ap
pt_Customer NOT NULL;

Table altered.

SQL> ALTER TABLE Appointment MODIFY Appointment_Date CONSTRAINT
NN_Appt_Date NOT NULL;

Table altered.
```

```
SQL> ALTER TABLE Appointment_Details
  2  DROP CONSTRAINT CHK_Appt_Price;

Table altered.

SQL>  ALTER TABLE Appointment_Details ADD CONSTRAINT CHK_Appt_Quantity CHECK(Quantity > 0);

Table altered.

SQL> ALTER TABLE Appointment_Details ADD CONSTRAINT CHK_Appt_Price CHECK(Price > 0);

Table altered.

SQL>  ALTER TABLE Appointment ADD CONSTRAINT CHK_Appt_Amount CHECK(Total_Amount > 0);

Table altered.
```

```
SQL> ALTER TABLE Appointment ADD CONSTRAINT CHK_Appt_Status CHECK(Appointment_Status IN ('PENDING', 'CONFIRMED',
'COMPLETED'));

Table altered.
```

Insert Values :

APPOINTMENT table

```
SQL> INSERT INTO Appointment
  2  VALUES ('A006', 'CU-1006', DATE '2026-10-29', 750, 'PENDING');

1 row created.

SQL>
SQL> INSERT INTO Appointment
  2  VALUES ('A007', 'CU-1007', DATE '2026-05-30', 1800, 'COMPLETED');

1 row created.

SQL>
SQL> INSERT INTO Appointment
  2  VALUES ('A008', 'CU-1008', DATE '2026-10-31', 950, 'PENDING');

1 row created.

SQL>
SQL> INSERT INTO Appointment
  2  VALUES ('A009', 'CU-1009', DATE '2026-02-01', 1100, 'PENDING');

1 row created.
```

```
SQL> select * from Appointment;

APPOINT CUSTOMER_ID          APPOINTME TOTAL_AMOUNT APPOINTMENT_STATUS
------- -------------------- --------- ------------ --------------------
A001    CU-1001              24-OCT-26          999 COMPLETED
A002    CU-1002              25-JAN-26         1200 PENDING
A004    CU-1004              27-DEC-26          800 COMPLETED
A005    CU-1005              28-JUN-26         2000 PENDING
A010    CU-1010              02-APR-26         2500 COMPLETED
A003    CU-1003              26-OCT-26         1500 PENDING
A006    CU-1006              29-OCT-26          750 PENDING
A007    CU-1007              30-MAY-26         1800 COMPLETED
A008    CU-1008              31-OCT-26          950 PENDING
A009    CU-1009              01-FEB-26         1100 PENDING

10 rows selected.
```

APPOINTMENT DETAILS table:

```
SQL> INSERT INTO Appointment_Details VALUES('&Appointment_Detail_ID','&Appointment
_ID','&Service_ID',&Quantity,&Price);
Enter value for appointment_detail_id: AD001
Enter value for appointment_id: A001
Enter value for service_id: SRV001
Enter value for quantity: 1
Enter value for price: 999
old   1: INSERT INTO Appointment_Details VALUES('&Appointment_Detail_ID','&Appoint
ment_ID','&Service_ID',&Quantity,&Price)
new   1: INSERT INTO Appointment_Details VALUES('AD001','A001','SRV001',1,999)

1 row created.
SQL> SELECT * FROM Appointment_Details;

APPOINT APPOINT SERVICE   QUANTITY      PRICE
------- ------- ------- ---------- ----------
AD001   A001    SRV001           1        999
AD002   A002    SRV002           2        650
AD003   A003    SRV002           3        750
AD004   A004    SRV004           2        549
AD005   A005    SRV005           2        899
AD006   A006    SRV001           4        899
AD007   A007    SRV003           2       1299
AD008   A008    SRV004           5       1499
AD009   A009    SRV005           3       1099
AD010   A010    SRV001           2        589

10 rows selected.
```

Report 1: Customer Appointment History

This report displays the appointment history of customers. It combines customer information with their appointment details.

Description

The query joins the CUSTOMER and APPOINTMENT tables using CUSTOMER_ID.
It displays:

- Customer name
- Appointment ID
- Appointment date
- Total amount
- Appointment status

This report helps in viewing the complete booking history of each customer.

```
SQL> SELECT c.FIRST_NAME || ' ' || c.LAST_NAME AS CUSTOMER_NAME,
  2         a.APPOINTMENT_ID,
  3         a.APPOINTMENT_DATE,
  4         a.TOTAL_AMOUNT,
  5         a.APPOINTMENT_STATUS
  6  FROM CUSTOMER c
  7  JOIN APPOINTMENT a
  8  ON c.CUSTOMER_ID = a.CUSTOMER_ID
  9  ORDER BY c.FIRST_NAME, a.APPOINTMENT_DATE;

CUSTOMER_NAME                             APPOINT APPOINTME TOTAL_AMOUNT
----------------------------------------- ------- --------- ------------
APPOINTMENT_STATUS
--------------------
ANANYA  SURESH                            A003    26-OCT-26         1500
PENDING

HARI KUMAR                                A008    31-OCT-26          950
PENDING

KAVYA RAO                                 A002    25-JAN-26         1200
PENDING


CUSTOMER_NAME                             APPOINT APPOINTME TOTAL_AMOUNT
----------------------------------------- ------- --------- ------------
APPOINTMENT_STATUS
--------------------
MEERA IYER                                A001    24-OCT-26          999
COMPLETED

MOSHAMI JUN                               A007    30-MAY-26         1800
COMPLETED

NILLA VEERA                               A010    02-APR-26         2500
COMPLETED


CUSTOMER_NAME                             APPOINT APPOINTME TOTAL_AMOUNT
----------------------------------------- ------- --------- ------------
APPOINTMENT_STATUS
--------------------
NISHA KUMARI                              A005    28-JUN-26         2000
PENDING

PRIYA NAIR                                A004    27-DEC-26          800
COMPLETED

SRI VALLI                                 A009    01-FEB-26         1100
PENDING


CUSTOMER_NAME                             APPOINT APPOINTME TOTAL_AMOUNT
----------------------------------------- ------- --------- ------------
APPOINTMENT_STATUS
--------------------
VARSHA RAJAN                              A006    29-OCT-26          750
PENDING


10 rows selected.
```

Report 2: Service-wise Appointment Report

This report provides information about the number of appointments and total quantity booked for each service.

Description

The query connects the SERVICE and APPOINTMENT_DETAILS tables.

It calculates:

- Service name
- Number of appointments
- Total quantity booked

The COUNT() function is used to count appointments, while SUM() calculates the total quantity.

The result is arranged according to the total quantity booked.

```
SQL> SELECT s.SERVICE_NAME,
  2         COUNT(ad.APPOINTMENT_ID) AS APPOINTMENT_COUNT,
  3         SUM(ad.QUANTITY) AS TOTAL_QUANTITY
  4  FROM SERVICE s
  5  JOIN APPOINTMENT_DETAILS ad
  6  ON s.SERVICE_ID = ad.SERVICE_ID
  7  GROUP BY s.SERVICE_NAME
  8  ORDER BY TOTAL_QUANTITY DESC;

SERVICE_NAME                                       APPOINTMENT_COUNT
-------------------------------------------------- -----------------
TOTAL_QUANTITY
--------------
CLASSIC MAINCURE                                                   3
             7

CHROME NAIL ART                                                    2
             7

CLASSIC PEDICURE                                                   2
             5


SERVICE_NAME                                       APPOINTMENT_COUNT
-------------------------------------------------- -----------------
TOTAL_QUANTITY
--------------
SPA PEDICURE                                                       2
             5

NAIL EXTENSIONS                                                    1
             2
```

Report 3: Customer Appointment Analysis

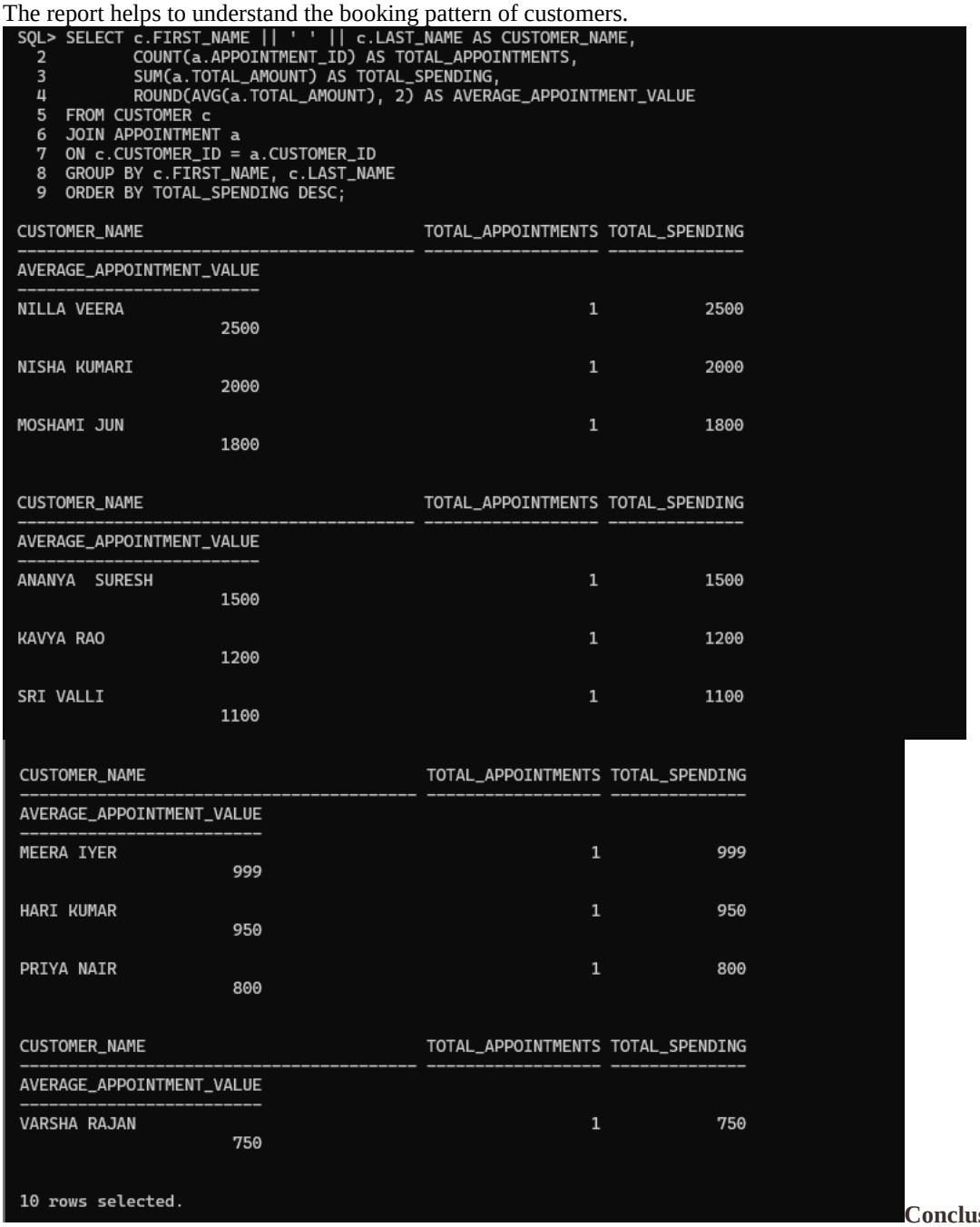
This report analyzes the booking activity of customers.
It shows the total number of appointments, total amount spent and average appointment value for each customer.

Description

The query uses aggregate functions to analyze customer bookings.

- COUNT() calculates the total number of appointments.
- SUM() calculates the total spending.
- AVG() calculates the average appointment value.
- ROUND() displays the average value up to two decimal places.

The report helps to understand the booking pattern of customers.



```
SQL> SELECT c.FIRST_NAME || ' ' || c.LAST_NAME AS CUSTOMER_NAME,
  2         COUNT(a.APPOINTMENT_ID) AS TOTAL_APPOINTMENTS,
  3         SUM(a.TOTAL_AMOUNT) AS TOTAL_SPENDING,
  4         ROUND(AVG(a.TOTAL_AMOUNT), 2) AS AVERAGE_APPOINTMENT_VALUE
  5  FROM CUSTOMER c
  6  JOIN APPOINTMENT a
  7  ON c.CUSTOMER_ID = a.CUSTOMER_ID
  8  GROUP BY c.FIRST_NAME, c.LAST_NAME
  9  ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_NAME                          TOTAL_APPOINTMENTS TOTAL_SPENDING
-------------------------------------- ------------------ --------------
AVERAGE_APPOINTMENT_VALUE
-------------------------
NILLA VEERA                                             1           2500
                     2500

NISHA KUMARI                                            1           2000
                     2000

MOSHAMI JUN                                             1           1800
                     1800


CUSTOMER_NAME                          TOTAL_APPOINTMENTS TOTAL_SPENDING
-------------------------------------- ------------------ --------------
AVERAGE_APPOINTMENT_VALUE
-------------------------
ANANYA  SURESH                                          1           1500
                     1500

KAVYA RAO                                               1           1200
                     1200

SRI VALLI                                               1           1100
                     1100


CUSTOMER_NAME                          TOTAL_APPOINTMENTS TOTAL_SPENDING
-------------------------------------- ------------------ --------------
AVERAGE_APPOINTMENT_VALUE
-------------------------
MEERA IYER                                              1            999
                      999

HARI KUMAR                                              1            950
                      950

PRIYA NAIR                                              1            800
                      800


CUSTOMER_NAME                          TOTAL_APPOINTMENTS TOTAL_SPENDING
-------------------------------------- ------------------ --------------
AVERAGE_APPOINTMENT_VALUE
-------------------------
VARSHA RAJAN                                            1            750
                      750


10 rows selected.
```

Conclusion

In this week, the APPOINTMENT and APPOINTMENT_DETAILS tables were created for the Nailverse database. The required primary key, foreign key, NOT NULL and CHECK constraints were applied to maintain data accuracy. The relationships between customers, appointments and services were also established. These reports help to understand customer booking history, service demand and customer spending patterns. Thus, the appointment management part of the Nailverse database was successfully implemented.